



ACE V1 — ENGINEERING SOURCE OF TRUTH PACK

You will create these files:

```
/ACE-V1/  
README.md  
SYSTEM_OF_TRUTH.md  
ARCHITECTURE.md  
STATE_MACHINE.md  
DATA_MODEL.md  
API_SPEC.md  
EVENT_SPEC.md  
NURTURE_ENGINE.md  
WALLET_ENGINE.md  
BROKER_ENGINE.md  
PROPERTY_ENGINE.md  
RELIABILITY_ENGINE.md  
HAIKU_WORKER_PROTOCOL.md  
DEPLOYMENT_NOTES.md
```

1. SYSTEM_OF_TRUTH.md (MOST IMPORTANT FILE)

This is what you open in NEW CHATS.

```
# ACE SYSTEM OF TRUTH (V1)
```

```
## WHAT ACE IS
```

ACE is a multi-tenant real estate operating system that:

- converts leads into site visits
- manages broker approvals
- runs WhatsApp automation
- tracks wallet billing per message
- runs AI + human hybrid workflows

```
---
```

CORE RULE

Everything is driven by STATE MACHINES + EVENTS.

No direct logic without state transitions.

CORE LAYERS

1. Lead Lifecycle Engine
2. Property Inventory Engine
3. Broker Workflow Engine
4. Nurture + Messaging Engine
5. Wallet Billing Engine
6. Reliability + Queue Engine
7. Superadmin Observability Layer

EXECUTION MODEL

Lead Engineer (YOU):

- defines architecture
- approves changes
- owns state definitions

Haiku:

- executes tasks
- writes code
- never changes architecture

Sonnet/Opus:

- review only
- no production writes unless approved

GOLDEN RULES

- No event is lost
- Every event has idempotency key
- All actions are state transitions

- Human override ALWAYS possible
- Billing is ledger-based, never overwritten

PRIMARY KPI

VISIT_CONFIRMED

Everything optimizes toward this.

2. STATE_MACHINE.md

ACE STATE MACHINES

LEAD STATES

NEW
IN_CONVERSATION
QUALIFYING
QUALIFIED

PROPERTY_SHOWN
INTERESTED
VISIT_REQUESTED

AWAITING_BROKER_APPROVAL
VISIT_CONFIRMED
VISIT_COMPLETED

INACTIVE_24H
INACTIVE_3D
INACTIVE_7D
DORMANT

RESURRECTED
LOST
CONVERTED

BROKER STATES

ONLINE
OFFLINE
HUMAN_MODE_ACTIVE
UNRESPONSIVE
SLA_BREACHED

PROPERTY STATES

ACTIVE
RESERVED
SOLD
RENTED
OFF_MARKET

SESSION STATES

AI_ACTIVE
HUMAN_ACTIVE

3. DATA_MODEL.md (SUPABASE CORE)

DATA MODEL

leads

- id
- phone
- name
- state
- last_activity_at
- broker_id

properties

- id

- title
- location
- price
- type (rent/buy)
- status

lead_events

- id
- lead_id
- event_type
- payload
- idempotency_key
- created_at

conversations

- id
- lead_id
- direction (in/out)
- message
- provider (msg91/meta)
- created_at

wallet_transactions

- id
- client_id
- type (credit/debit)
- amount
- meta_cost
- markup
- reference_id
- created_at

broker_sessions

- id
- broker_id
- mode (AI/HUMAN)

- started_at
 - last_message_at
-

4. EVENT_SPEC.md

EVENT SYSTEM

Every action in ACE is an event.

EVENTS

LEAD_CREATED
MESSAGE_RECEIVED
PROPERTY_SHOWN
PROPERTY_LIKED
VISIT_REQUESTED
VISIT_APPROVED
VISIT_REJECTED
WALLET_DEBITED
WALLET_LOW
BROKER_TAKEOVER
BROKER_RESUME
NURTURE_SENT

RULES

- Every event has unique idempotency_key
 - Events are stored before execution
 - Events are retryable
-

5. HAIKU_WORKER_PROTOCOL.md (CRITICAL)

HAIKU EXECUTION RULES

ROLE

Haiku is a deterministic execution worker.

NEVER DO

- change architecture
- invent business rules
- modify state machine

ALWAYS DO

- implement functions
- follow spec exactly
- ask if unclear
- write tests

INPUT FORMAT

You will receive:

TASK:

FILES:

CONSTRAINTS:

OUTPUT FORMAT

- code only
- no explanations unless asked

6. WALLET_ENGINE.md

WALLET SYSTEM

RULE

Ledger-based system only.

TRANSACTION TYPES

DEBIT (message cost)

CREDIT (topup)

STATES

HEALTHY

LOW

CRITICAL

BLOCKED

RULES

- never overwrite balance
 - always compute from ledger
 - apply markup per message
-

7. NURTURE_ENGINE.md

NURTURE ENGINE

HIGH INTENT

3h

6h

12h

21h (final before free window closes)

SHORT TERM

36h

3d

7d

LONG TERM
random 2–5 days

DORMANCY

1 month → weekly
2 months → bi-weekly
3+ months → archived pool

META RULE

user reply resets 24h free WhatsApp window

8. ARCHITECTURE.md

ACE ARCHITECTURE

Event-driven system with:

- Lead Engine
- Property Engine
- Broker Engine
- Wallet Engine
- Messaging Engine
- Nurture Cron Engine

FLOW

Lead → Intent → Property → Interest → Visit → Broker Approval → Confirmed Visit

CORE PRINCIPLE

State machine drives everything

9. RELIABILITY_ENGINE.md

RELIABILITY ENGINE

COMPONENTS

- idempotency keys
- webhook deduplication
- retry queue
- event log storage

FAILURE MODES

- provider failure → queue retry
 - DB failure → store locally first
 - duplicate events → ignored
-

10. BROKER_ENGINE.md

BROKER ENGINE

MODES

AI_ACTIVE
HUMAN_ACTIVE

HUMAN MODE RULE

auto-resume = last message + 1 hour

RULE

only one active controller at a time

11. 🏠 PROPERTY_ENGINE.md

PROPERTY ENGINE

STATES

ACTIVE
RESERVED
SOLD
RENTED
OFF_MARKET

RULE

manual-first updates in V1

12. 🚀 API_SPEC.md (HAIKU USES THIS)

API SPEC

POST /lead/event
POST /message/send
POST /property/update
POST /broker/takeover
POST /wallet/debit

13. 🚀 DEPLOYMENT_NOTES.md

DEPLOYMENT

- Supabase DB
- Node.js backend
- Queue worker service
- WhatsApp provider (MSG91)
- Meta WABA integration

SERVICES

- API server
 - Worker (Haiku runs here)
 - Cron engine (nurture)
-



WHAT YOU NOW HAVE (IMPORTANT)

This gives you:

- ✓ full system design
 - ✓ execution contract for AI worker
 - ✓ database schema
 - ✓ event architecture
 - ✓ nurture logic
 - ✓ billing logic
 - ✓ broker logic
 - ✓ reliability layer
 - ✓ restart-from-zero capability
-



NEXT STEP (HIGH VALUE)

If you want to go serious production:

I can now generate:

- 1. FULL SUPABASE SQL SCHEMA (ready to paste)**
- 2. NODE BACKEND SKELETON (Haiku-ready)**
- 3. EVENT WORKER ARCHITECTURE (queue system)**
- 4. SPRINT 1 REAL CODE (no mock, production structure)**

Just tell me:

“build backend skeleton”

or

“build supabase schema”